Software Requirements Specification for Project Proxi

Team #23, Project Proxi
Savinay Chhabra
Amanbeer Singh Minhas
Gourob Podder
Ajay Singh Grewal

April 6, 2026

Contents

1 Purpose of the Project **vii**

1.1 User Business . . . vii

1.2 Goals of the Project . . . vii

2 Naming Conventions and Terminology **viii**

2.1 Symbolic Constants . . . ix

2.2 Glossary of All Terms, Including Acronyms, Used by Stakeholders Involved in the Project . . . x

3 Stakeholders **xi**

3.1 Client . . . xi

3.2 Customer . . . xi

3.3 Other Stakeholders . . . xii

3.4 Hands-On Users of the Project . . . xii

3.5 Personas . . . xii

3.6 Priorities Assigned to Users . . . xiv

3.7 How Personas Affect the Requirements . . . xiv

3.8 User Participation . . . xv

3.9 Maintenance Users and Service Technicians . . . xv

4 Mandated Constraints **xvi**

4.1 Solution Constraints . . . xvi

4.2 Implementation Environment of the Current System . . . xvii

4.3 Partner or Collaborative Applications . . . xviii

4.4 Off-the-Shelf Software . . . xviii

4.5 Anticipated Workplace Environment . . . xix

4.6 Schedule Constraints . . . xxi

4.7 Budget Constraints . . . xxii

4.8 Enterprise Constraints . . . xxii

5 Relevant Facts and Assumptions **xxii**
5.1 Relevant Facts xxii
5.2 Business Rules xxii
5.3 Assumptions xxiii

6 The Scope of the Work **xxiii**
6.1 The Current Situation xxiii
6.2 The Context of the Work xxiv
6.3 Work Partitioning xxv
6.4 Specifying a Business Use Case (BUC) xxv

7 Business Data Model and Data Dictionary **xxvii**
7.1 Business Data Model xxviii
7.2 Data Dictionary xxviii

8 The Scope of the Product **xxx**
8.1 Product Boundary xxxi
8.2 Product Use Case Table xxxi
8.3 Individual Product Use Cases (PUC's) xxxii
8.4 System State Diagram xxxiv

9 Functional Requirements **xxxiv**
9.1 Functional Requirements xxxv
9.2 Traceability Summary xxxvii
9.3 Formalization of Interaction Safety Protocol xxxviii

10 Look and Feel Requirements **xxxix**
10.1 Appearance Requirements xxxix
10.2 Style Requirements xli

11 Usability and Humanity Requirements **xli**
11.1 Ease of Use Requirements xlii

11.2 Personalization and Internationalization Requirements xliii
11.3 Learning Requirements xliii
11.4 Understandability and Politeness Requirements xliv
11.5 Accessibility Requirements xlv

12 Performance Requirements **xlv**
12.1 Speed and Latency Requirements xlvi
12.2 Safety-Critical Requirements xlvi
12.3 Precision or Accuracy Requirements xlvii
12.4 Robustness or Fault-Tolerance Requirements xlviii
12.5 Capacity Requirements xlviii
12.6 Scalability or Extensibility Requirements xlix
12.7 Longevity Requirements l

13 Operational and Environmental Requirements **l**
13.1 Expected Physical Environment li
13.2 Wider Environment Requirements li
13.3 Requirements for Interfacing with Adjacent Systems lii
13.4 Productization Requirements lii
13.5 Release Requirements liii

14 Maintainability and Support Requirements **liv**
14.1 Maintenance Requirements liv
14.2 Supportability Requirements liv
14.3 Adaptability Requirements lv

15 Security Requirements **lv**
15.1 Access Requirements lvi
15.2 Integrity Requirements lvi
15.3 Privacy Requirements lvii
15.4 Audit Requirements lvii

15.5 Immunity Requirements lviii

16 Cultural Requirements **lviii**

16.1 Cultural Requirements lix

17 Compliance Requirements **lix**

17.1 Legal Requirements lix

17.2 Standards Compliance Requirements lx

18 Traceability from Requirements to Use Cases **lx**

19 Open Issues **lxii**

20 Off-the-Shelf Solutions **lxii**

20.1 Ready-Made Products lxii

20.2 Reusable Components lxiii

20.3 Products That Can Be Copied lxiii

21 New Problems **lxiii**

21.1 Effects on the Current Environment lxiv

21.2 Effects on the Installed Systems lxiv

21.3 Potential User Problems lxiv

21.4 Limitations in the Anticipated Implementation Environment That May Inhibit the New Product lxv

21.5 Follow-Up Problems lxv

22 Tasks **lxv**

22.1 Project Planning lxv

22.2 Planning of the Development Phases lxvi

23 Migration to the New Product **lxvi**

23.1 Requirements for Migration to the New Product lxvi

23.2 Data That Has to be Modified or Translated for the New System lxvii

24 Costs **lxvii**

25 User Documentation and Training **lxvii**

25.1 User Documentation Requirements lxvii

25.2 Training Requirements lxviii

26 Waiting Room **lxviii**

27 Ideas for Solution **lxix**

28 Reflections **lxxi**

28.1 Amanbeer Minhas Reflection lxxi

28.2 Gourob Podder Reflection lxxiii

28.3 Ajay Grewal Reflection lxxv

28.4 Savinay Chhabra Reflection lxxvi

Revision History

Date	Ver.	Notes
10/10/2025	1.0	Added initial SRS.
05/04/2026	1.1	Added introductory text for major sections.
		Moved repeated numeric thresholds into a symbolic constants section.
		Rewrote project goals to be higher level.
		Reordered glossary entries alphabetically.
		Corrected spelling and wording issues.
		Added rationale for key constraints.
		Labeled assumptions as A1, A2, etc.
		Clarified how personas affect requirement priorities.
		Improved spacing and readability in Section 8.3.
		Added dedicated traceability from requirements to use cases.
		Expanded schedule and task-planning detail.
		Added a formalized safety interaction protocol and a system state diagram subsection.

1 Purpose of the Project

1.1 User Business

Proxi is an AI-powered desktop assistant designed to make everyday computer use more accessible, efficient, and dependable. It allows people to operate a computer through natural speech and text rather than requiring every task to be completed through traditional mouse- and keyboard-heavy workflows. The product is especially relevant for users who face barriers with complex graphical interfaces, older adults who benefit from simpler interaction paths, and users with temporary or permanent accessibility needs who need more flexible control of the desktop environment [World Health Organization, 2023, World Wide Web Consortium (W3C), 2023].

From a business perspective, Proxi addresses two closely related needs. The first is accessibility: many users cannot consistently rely on precise pointing, shortcut memorization, or dense interface navigation, so a natural-language assistant can reduce friction and increase independence. The second is productivity: even experienced users often want faster, lower-friction workflows for repetitive desktop tasks such as opening applications, searching for files, drafting messages, or triggering common actions. By supporting both groups, Proxi can serve as a practical daily-use assistant rather than a novelty interface.

Proxi's business value comes from converting user intent into safe, traceable computer actions. Instead of forcing users to translate their goals into technical steps, the system handles interpretation, planning, confirmation, and execution in a way that preserves user control. This reduces the digital divide, improves confidence for novice users, and gives power users a more direct way to complete routine work while still keeping destructive or risky actions visible and confirmable.

1.2 Goals of the Project

The goals in this subsection are intentionally high level. Quantitative targets that serve as fit criteria are centralized in the symbolic constants table in Section 2.1 so that the same values can be referenced consistently throughout the document. This separation helps keep the goals stable even when implemen-

tation details evolve, while still preserving measurable acceptance criteria in later requirement sections. It also improves traceability by ensuring that performance, usability, and safety thresholds are defined once and reused uniformly across functional requirements, non-functional requirements, and verification activities.

G-1 (Accessibility)	Proxi shall improve computer access for users who face barriers with keyboards, mice, and complex user interfaces.
G-2 (Independent Task Completion)	Proxi shall allow users to complete a meaningful set of everyday computer tasks through voice or text interaction.
G-3 (Safety and Trust)	Proxi shall support safe task execution through clear feedback, explicit confirmation for risky actions, and traceable results.
G-4 (Usability)	Proxi shall be simple enough for new users to learn quickly and useful enough for experienced users to adopt in everyday work.
G-5 (Extensibility)	Proxi shall provide a modular foundation that can be extended with new tools, interaction modes, and accessibility improvements.
G-6 (Stretch Goals)	Stretch goals include stronger personalization, offline support, multimodal input, and broader language coverage.

2 Naming Conventions and Terminology

This section defines project terms, abbreviations, and shared constants. The glossary is ordered alphabetically to improve lookup, and the symbolic constants centralize repeated numeric thresholds so the same values can be used consistently across goals, requirements, and fit criteria.

2.1 Symbolic Constants

This section defines the main symbolic constants used throughout this document. These constants centralize recurring thresholds so values are defined once and referenced consistently across goals, functional requirements, non-functional requirements, and fit criteria. Using symbolic names instead of repeating raw numbers also improves maintainability: if a target changes, it can be updated in one place without introducing inconsistencies in downstream sections.

Table 1: Symbolic Constants

Name	**Value**	**Meaning**
`APP_CPU_MAX`	30%	Maximum CPU use during normal operation.
`APP_LOAD_TIME`	200 ms	Maximum time to load a new feature.
`INDOOR_NOISE_MAX`	65 dB	Moderate indoor noise limit.
`MOD_NOISE_ACC_MIN`	80%	Minimum accuracy in moderate indoor noise.
`QUIET_ACC_MIN`	90%	Minimum recognition accuracy in quiet conditions.
`QUIET_RESP_MAX`	2.0 s	Maximum response-start time in quiet conditions.
`ROLLBACK_MAX`	60 s	Maximum rollback time after update failure.
`SESS_DURATION_MIN`	4 h	Minimum continuous operation target.
`TASK_SUCCESS_MIN`	85%	Minimum end-to-end task success rate.
`TASK_TIME_MAX`	3.0 s	Maximum time for common local actions.
`USER_SAT_MIN`	4.0/5.0	Minimum usability satisfaction score.
`USABILITY_GAIN_MIN`	20%	Minimum improvement over baseline workflow.

2.2 Glossary of All Terms, Including Acronyms, Used by Stakeholders Involved in the Project

Term	Definition
Accessibility	Usable by people with a wide range of abilities and disabilities.
Action	A single execution of a tool with validated parameters.
AI Agent	A software agent that uses artificial intelligence to carry out tasks.
Audit Log	A record of actions taken by the system.
Command	An instruction given by the user to the system.
Confirmation Gate	An approval prompt shown before risky actions.
LLM	Large Language Model.
MCP	Model Context Protocol.
MCP Client	An AI agent that connects to an MCP server to use tools.
MCP Server	A local server that exposes tools through MCP.
OS Automation Tools	Software that allows the system to interact with the operating system.
Permission Scope	The level of access granted to the system for completing actions.
PII	Personally Identifiable Information.
Plan	An ordered sequence of actions used to fulfill a user intent.
POLP	Principle of Least Privilege.
Sandbox	A secure environment in which code can run without affecting the rest of the system.
STT	Speech-to-Text.
Token	A unit of text used by language models.
TTS	Text-to-Speech.
User Interface	The visual and interactive elements through which a user engages with the system.

3 Stakeholders

This section identifies the people and groups who influence scope, implementation, validation, and long-term use of the product. The goal is to make decision authority, user impact, and feedback pathways explicit so requirement priorities remain defensible throughout the project lifecycle.

3.1 Client

The clients for this project are the SFWRENG 4G06A Capstone teaching team at McMaster University, including the course instructor and the assigned teaching assistants.

Role: They act as the academic product owners for scope, quality, and acceptance decisions.

Decision Authority: They approve major requirement changes, milestone outcomes, and final deliverable readiness.

Primary Success Criteria: The delivered system demonstrates strong accessibility alignment, usable interaction flow, and acceptable engineering rigor for a capstone context.

3.2 Customer

Our customers are the end users and organizations that will deploy Proxi to enable inclusive and more efficient computer use:

1. **Educational institutions** such as libraries, computer labs, and accessibility services that need dependable low-friction computer access support.
2. **Healthcare and community organizations** supporting users with motor, vision, or hearing challenges who benefit from voice-first and guided interaction.
3. **General consumers and power users** who value faster workflows, reduced interaction overhead, and mixed-modality control in daily desktop tasks.

Across these customer groups, the most important shared outcomes are **safe task completion**, **clear feedback**, and **predictable behavior**.

3.3 Other Stakeholders

Other stakeholders include:

1. **Team Proxi (development team):** Responsible for requirements analysis, architecture, implementation, testing, documentation, and integration quality.
2. **Course staff:** Provide assessment criteria, risk-oriented feedback, and milestone governance.
3. **Accessibility advisors (if engaged):** Contribute practical guidance on WCAG-aligned decisions and assistive-technology compatibility.
4. **Pilot participants:** Provide early evidence of usability barriers, task-flow friction, and communication clarity.

3.4 Hands-On Users of the Project

Primary hands-on users who will directly interact with Proxi:

1. **Accessibility-focused users:** People with motor impairments or temporary and situational limitations who need hands-free, low-complexity, and error-tolerant interaction.
2. **General and power users:** Users seeking faster and more natural workflows through voice or text, while retaining keyboard and mouse fallback paths.

These groups are expected to use the same core capabilities, but they place different emphasis on speed, guidance density, and confirmation behavior.

3.5 Personas

1. **P1: Amrita (72) - Elderly User**

   Profile: Amrita is a retired teacher with moderate digital literacy who uses her desktop for personal communication, banking, government forms, and telehealth appointments.

Primary Tasks: Read and respond to email, open documents from family members, complete simple web forms, and join scheduled video calls.

Pain Points: Small controls, dense interfaces, and multi-step flows increase cognitive load. She often loses track of where she is in a process and worries about making irreversible mistakes.

Accessibility and Usability Needs: Large and readable text, plain-language prompts, explicit progress feedback, low-jargon error recovery, and clear confirmation before risky actions.

Success Indicators: She can complete common tasks without external help, understands system status at each step, and confidently recovers from mistakes.

2. **P2: Leo (21) - User with Motor Disability**

   Profile: Leo is a university student with a motor impairment that makes prolonged keyboard and mouse interaction painful and slow.

   Primary Tasks: Open and navigate course documents, switch between browser tabs and note-taking tools, draft assignment text, and manage file organization during study sessions.

   Pain Points: Repetitive fine-motor interactions, shortcut-heavy workflows, and precise cursor targeting create fatigue and reduce productivity. Frequent context switching further amplifies input overhead.

   Accessibility and Usability Needs: Reliable voice-first execution, complete keyboard-free coverage of core actions, captioned feedback, predictable command structure, and low-latency responses for uninterrupted work.

   Success Indicators: He can complete end-to-end coursework workflows hands-free with minimal corrections and without dependency on another person.

3. **P3: Ari (28) - Power User**

   Profile: Ari is a productivity-focused knowledge worker who manages high task volume across spreadsheets, email, messaging, and browser-based systems.

Primary Tasks: Trigger repeated multi-step workflows, retrieve files quickly, draft routine communications, and coordinate tasks across several applications in short cycles.

Pain Points: Workflow fragmentation, repeated navigation, and command recall overhead interrupt concentration and reduce throughput.

Accessibility and Usability Needs: Fast execution, concise confirmations, efficient follow-up commands, and reusable patterns that reduce repetitive interaction cost while preserving guardrails for risky operations.

Success Indicators: He completes repeated workflows in fewer interaction steps, with lower context-switch overhead and sustained trust in system safety.

Persona Design Implication: The interface must balance **clarity-first interaction** for users like Amrita and Leo with **efficiency-first flow** for users like Ari, without compromising safety controls.

3.6 Priorities Assigned to Users

Primary Priority Group: Accessibility-focused users, including elderly users like Amrita and users with motor impairments like Leo. Their needs define baseline acceptance for usability, interaction clarity, caption support, voice control, and safety confirmation.

Secondary Priority Group: General and power users like Ari. Their needs mainly influence efficiency, reduced interaction steps, and high-frequency workflow support after accessibility-critical requirements are satisfied.

3.7 How Personas Affect the Requirements

The personas directly influence requirement priorities. P1 Amrita and P2 Leo drive the highest-priority requirements for simple language, low-friction task completion, captions, scalable text, voice-first operation, and explicit safety confirmation. P3 Ari mainly influences secondary requirements related to speed, efficient task flow, and support for repeated multi-step tasks.

In practical terms, persona influence is applied as follows:

1. **P1 and P2 constraints are treated as release-critical** for core interaction flows.
2. **P3-driven optimizations are treated as quality multipliers** once baseline accessibility and safety are satisfied.
3. **Conflicts are resolved in favor of safety and accessibility** when speed and guardrails compete.

3.8 User Participation

Estimated participation during the project will primarily involve end users and the development team. Accessibility-focused users are expected to participate for about one hour per week through remote or in-lab sessions focused on usability testing and feedback. General users and power users will contribute roughly one hour per week by taking part in efficiency and performance testing of new features.

Participation Objectives: Collect actionable feedback on command clarity, confirmation understandability, error recovery flow, and end-to-end task success.

Expected Outputs: Prioritized usability findings, requirement refinements, and targeted validation scenarios for upcoming milestones.

3.9 Maintenance Users and Service Technicians

The primary maintenance users for this project will be the development team. During the capstone project, they will be responsible for identifying and fixing issues, releasing updates and patches, and ensuring that the system continues to function as expected.

Maintenance Scope: Bug triage, integration fixes, dependency upkeep, and documentation updates for installation and troubleshooting.

Operational Goal: Preserve reliability and traceability across revisions while minimizing regression risk in accessibility and safety-critical behaviors.

4 Mandated Constraints

This section records external constraints and important project-level decisions that bound the solution space. Each constraint includes a short rationale so it is clear whether it comes from the project context, target environment, or a team decision.

4.1 Solution Constraints

ID	**SOL-01**
Requirement	The application shall run on standard consumer hardware and shall not require specialized assistive hardware.
Rationale	The target users are expected to use ordinary personal computers at home, school, or work.
Fit Criterion	The application installs and operates on ordinary laptops or desktops with a standard microphone, speakers, display, and keyboard.
Priority	High

ID	**SOL-02**
Requirement	The application shall operate without requiring a Proxi-specific user account or sign-up flow.
Rationale	Reducing setup friction is important for accessibility, first-time use, and privacy.
Fit Criterion	A user can install and run Proxi without creating a Proxi-managed account.
Priority	High

4.2 Implementation Environment of the Current System

ID	**IMECS-01**
Requirement	The application shall run on mainstream desktop operating systems such as Windows and macOS.
Rationale	These are the primary environments available to the intended users during the capstone project.
Fit Criterion	The application builds and runs on supported Windows and macOS test machines.
Priority	High

ID	**IMECS-02**
Requirement	The application shall rely on standard input and output devices such as microphones, speakers, keyboards, and displays.
Rationale	The product is intended to be deployable without extra equipment.
Fit Criterion	All core user flows are executable using only standard desktop peripherals.
Priority	High

ID	**IMECS-03**
Requirement	The implementation may depend on a web-based or JavaScript-based application stack and on MCP-compatible libraries when this simplifies delivery.
Rationale	This is a team design decision for implementation speed and ecosystem support, not a client-mandated constraint.
Fit Criterion	The selected implementation stack supports the required Proxi features within the project timeline.
Priority	Medium

4.3 Partner or Collaborative Applications

ID	**PCA-01**
Requirement	The application may use external AI services through secure APIs for natural-language understanding, planning support, and response generation.
Rationale	External AI services can improve language coverage and implementation speed while preserving modularity.
Fit Criterion	When enabled, external AI calls use secure transport and return task-relevant outputs that integrate with the local execution flow.
Priority	Medium

ID	**PCA-02**
Requirement	Integration with external accessibility tools, including screen readers and related assistive technologies, may be supported to improve usability for older adults and users with disabilities.
Rationale	Compatibility with existing assistive ecosystems reduces adoption barriers and improves inclusiveness.
Fit Criterion	Supported workflows do not block core interactions when common assistive tools are active on target platforms.
Priority	Medium

4.4 Off-the-Shelf Software

ID	**OTSS-01**
Requirement	MCP-compatible infrastructure shall be used to provide modular tool discovery, invocation, and interface coordination.
Rationale	Using MCP-compatible components improves extensibility and reduces custom integration overhead.
Fit Criterion	Tools are registerable and invokable through the selected MCP integration path in the working build.
Priority	High

ID	**OTSS-02**
Requirement	Off-the-shelf JavaScript libraries and frameworks shall be used where appropriate to implement interactive elements and handle input and output operations.
Rationale	Mature libraries accelerate development and reduce implementation risk for standard interaction patterns.
Fit Criterion	Core UI and interaction flows are implemented using maintained third-party components with project-appropriate licenses.
Priority	High

ID	**OTSS-03**
Requirement	Off-the-shelf LLM services may be used for natural-language responses and reasoning capabilities; no specialized model training is required for baseline scope.
Rationale	Pretrained LLM services are sufficient for capstone-level baseline intent handling and response generation.
Fit Criterion	Baseline intent interpretation and response quality meet the relevant functional and usability targets without custom model training.
Priority	Medium

4.5 Anticipated Workplace Environment

ID	**AWE-01**
Requirement	The application is expected to run on users' personal computers in homes, schools, offices, and assisted-living facilities.
Rationale	The deployment context is primarily everyday desktop environments, not specialized labs.
Fit Criterion	Core workflows execute in representative home, school, and office test environments on supported desktop systems.
Priority	High

ID	**AWE-02**
Requirement	The application shall be usable primarily on Windows and macOS operating systems.
Rationale	These operating systems represent the primary target environments for project users.
Fit Criterion	Core features are demonstrated and validated on supported versions of Windows and macOS.
Priority	High

ID	**AWE-03**
Requirement	The system shall support users with limited technical experience, including older adults and people with disabilities, through a simple and intuitive interaction model.
Rationale	Ease of understanding is essential for inclusive adoption and safe use.
Fit Criterion	Usability sessions with novice and accessibility-focused users show completion of representative tasks with minimal facilitator support.
Priority	High

ID	**AWE-04**
Requirement	The application shall function on standard consumer computers with varying screen sizes, input methods, and processing power, without requiring specialized hardware.
Rationale	Hardware variability is expected in the target user population.
Fit Criterion	Core functionality remains available across representative low- to mid-range desktop hardware profiles.
Priority	Medium

ID	**AWE-05**
Requirement	Because background noise and poor acoustics can reduce voice performance, the application shall provide alternative input paths, including text and on-screen controls, when voice input is infeasible.
Rationale	Reliable fallback input is required to preserve task continuity in non-ideal listening conditions.
Fit Criterion	Representative tasks remain completable through non-voice input during noise-degraded test scenarios.
Priority	High

4.6 Schedule Constraints

Development will follow the planned project timeline while giving earlier attention to the highest-priority accessibility, safety, and core-task features. Table 2 summarizes the main external milestones.

Table 2: Project Timeline

Deliverable	**Deadline**
Req. Doc. and Hazard Analysis Revision 0	October 6, 2025
V&V Plan Revision 0	October 27, 2025
Design Document Revision -1	November 10, 2025
Design Document Revision 0	January 19, 2026
Revision 0 Demonstration	February 13, 2026
V&V Report and Extras Revision 0	March 9, 2026
Final Demonstration (Revision 1)	March 29, 2026
Final Documentation (Revision 1)	April 6, 2026
EXPO Demonstration	April 7, 2026

4.7 Budget Constraints

The project budget is limited to $500 in total expenditure. Of this, $125 will be provided by the CAS department for approved expenses.

4.8 Enterprise Constraints

There are no enterprise constraints beyond the need to satisfy course requirements and academic policies. The application may use any service or LLM as required as long as it adheres to the budget constraints.

5 Relevant Facts and Assumptions

This section separates established project facts, business rules, and explicit assumptions. Assumptions are labeled so they can be referenced by design, testing, and later revisions.

5.1 Relevant Facts

1. Proxi's main goal is to enable hands-free computer control through natural language commands for disabled users while also supporting users who want a more efficient workflow.
2. The system will run on desktop operating systems such as Windows and macOS and will interact with common applications such as web browsers, file browsers, calendar tools, and email tools.
3. Some actions can be high risk, such as deleting files or changing system settings, so the system must confirm with the user before performing such actions.

5.2 Business Rules

Proxi must get user consent before accessing or handling personal data. It must minimize the amount of data shared with external services and send only what is strictly needed to fulfill the user's request. Any command that

involves high risk, such as deleting data or changing system settings, must be confirmed before execution. All actions should be recorded in an audit log so that incorrect actions can be traced and, when possible, reversed. Tools may operate only within the permission scope explicitly granted by the user.

5.3 Assumptions

A1 (Hardware)
The user has a working computer with a microphone and audio output and can run the software on a modern desktop operating system.

A2 (Permissions)
The user can grant the permissions needed by the system, including microphone, accessibility, or automation permissions.

A3 (Internet)
The user has a reliable internet connection when using features that depend on remote LLM or web services.

A4 (Environment and Language)
The user is in an environment where the microphone can capture speech clearly and will speak in a supported language.

A5 (Off-the-Shelf Model Adequacy)
An off-the-shelf LLM is assumed to be adequate for the baseline command-understanding tasks in scope. This assumption will be checked through verification activities tied to functional accuracy, task success, and usability.

6 The Scope of the Work

This section describes the current problem context, the project's role within that context, the high-level partitioning of the work, and the main business use case that motivates the product.

6.1 The Current Situation

Desktop computing still assumes that users can reliably navigate dense graphical interfaces, operate precise pointer controls, and remember multi-step in-

teraction paths. For many users, this assumption does not hold in everyday practice.

Current Barriers:

- Users with motor impairments or temporary mobility limits may find keyboard and mouse interaction slow, fatiguing, or infeasible.
- Elderly and novice users can struggle with interface complexity, hidden settings, and jargon-heavy error messages.
- Voice assistants commonly used today provide limited task-execution depth for desktop workflows and often stop at shallow command handling.

As a result, many users cannot complete routine digital tasks with consistent independence, speed, and confidence. This creates a clear need for an assistant that is not only conversational, but also operationally useful across real desktop tasks.

6.2 The Context of the Work

This project addresses the above gap by building an AI-powered, desktop-oriented assistive platform that converts natural-language intent into safe and traceable computer actions.

Project Positioning:

- Proxi acts as an **intermediary layer** between the user and the operating environment.
- It accepts **speech and text input**, interprets intent, plans executable steps, and invokes tools through a modular integration model.
- It emphasizes **accessible interaction** and **safety-aware execution**, including explicit confirmation for risky actions and clear user feedback.

Within the scope of this capstone, the work focuses on practical desktop task support (for example opening applications, navigating files, and handling common communication workflows) rather than building full replacements for third-party software. This keeps the solution applicable, testable, and aligned with project constraints.

6.3 Work Partitioning

The work is partitioned into six coordinated workstreams so design, implementation, and validation can proceed in parallel while remaining traceable to requirements.

1. **Input and Recognition Layer**

   Captures user requests through voice and text channels and normalizes them into a consistent input format for downstream processing.

2. **Natural Language Understanding Layer**

   Interprets normalized input, resolves intent and entities, and produces structured task representations suitable for planning.

3. **Planning and Execution Layer**

   Generates safe execution plans and invokes modular tools to perform user-requested desktop actions within configured permission scope.

4. **Feedback and Output Layer**

   Communicates status, confirmations, errors, and outcomes through readable text and optional speech, with emphasis on clarity and recoverability.

5. **Accessibility and Usability Design**

   Defines interaction constraints and interface behavior for inclusive use, including readability, low-friction controls, and confirmation-oriented safety design.

6. **Evaluation and Testing**

   Verifies functional correctness, usability, performance, and safety behavior through scenario-based testing, stakeholder feedback, and milestone-aligned validation activities.

Together, these workstreams establish a clear delivery path from user input to validated task outcomes while preserving modularity for future extension.

6.4 Specifying a Business Use Case (BUC)

BUC-01: Intelligent Accessibility Assistant

Primary Actor: User with limited mobility, low computer literacy, or hands-busy use.

Supporting Actors: Operating system services, desktop applications, and connected tool integrations invoked through Proxi.

Goal: Complete desktop tasks through natural speech or text with minimal manual navigation while preserving user safety and control.

Trigger: The user issues a request in voice or text, for example "Open my email and draft a message to my doctor."

Preconditions:

- The user has access to a supported device with microphone, speakers, and input fallback options.
- The assistive system is installed, running, and has the required permissions for the requested action.
- If applicable, network connectivity is available for remote services used by the request.

Main Success Scenario:

1. The user submits a request using voice or text.
2. The system captures and normalizes the input.
3. The system identifies intent and required entities.
4. The system generates a task plan using available tools.
5. For risky actions, the system requests explicit confirmation.
6. The system executes the approved plan.
7. The system reports outcome status and next-step guidance in plain language (with speech and text when enabled).
8. The system records interaction and execution events for traceability.

Extensions and Exception Flows:

- **E1 - Ambiguous Request:** The system asks a concise clarification question and resumes planning after user disambiguation.

- **E2 - Missing Permission or Tool Access:** The system stops execution, explains the blocker, and provides corrective steps.
- **E3 - High-Risk Action Denied:** The system cancels the risky step, confirms no destructive change was made, and offers safe alternatives.
- **E4 - Execution Failure:** The system reports failure cause, logs diagnostic context, and suggests retry or fallback action.

Success Guarantee (Postconditions on Success):

- Requested task is completed or progressed to a valid next step.
- User receives clear completion feedback and any relevant follow-up options.
- Audit trail entries are persisted for command, plan, and outcome events.

Minimal Guarantee (Postconditions on Failure):

- No unconfirmed destructive action is executed.
- The user receives an understandable failure message and recovery guidance.
- Failure context is logged for support and debugging.

Business Value: This use case operationalizes inclusive, low-friction computer control by converting natural-language intent into safe, traceable desktop actions across everyday workflows.

7 Business Data Model and Data Dictionary

This section defines the core business entities and their main attributes. Figure 1 presents the business data model, and the subsequent tables define each entity.

7.1 Business Data Model

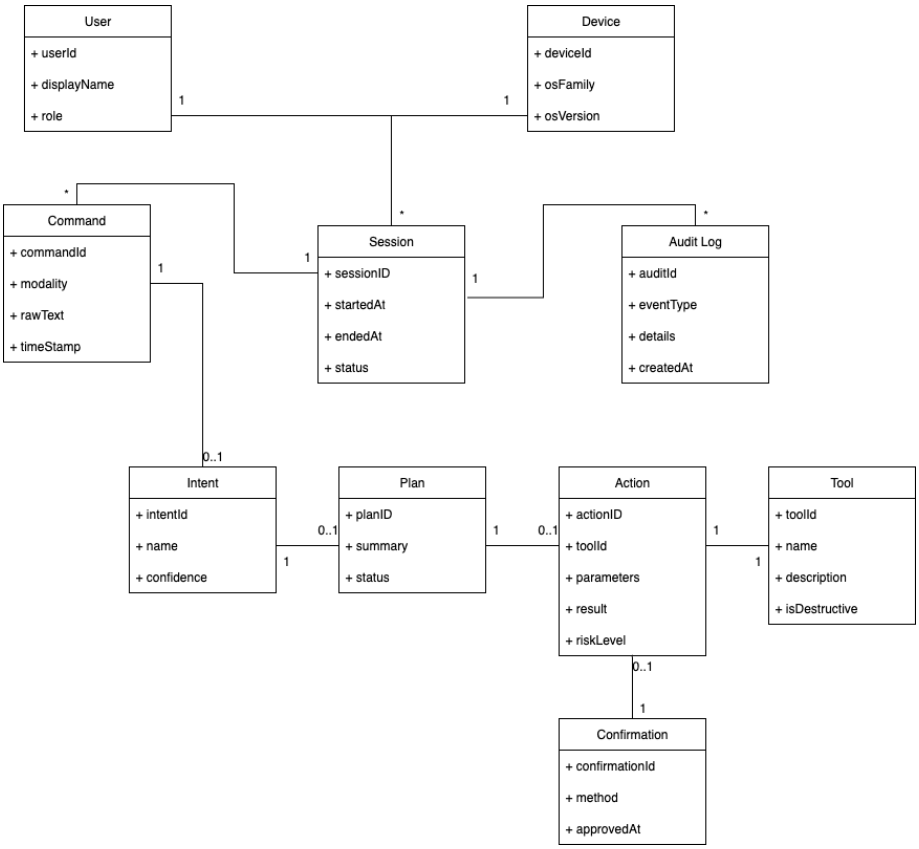

Figure 1: Business Data Model

7.2 Data Dictionary

User

Attribute	Type	Description
userId	UUID	Unique identifier for a user
displayName	String	Name shown in confirmations or UI
role	Enum	Authorization role

Session

Attribute	Type	Description
sessionId	UUID	Logical dialog or control session
startedAt / endedAt	Timestamp	Session start and end
status	Enum	Active, Ended, or Error

Command

Attribute	Type	Description
commandId	UUID	Captured user request
modality	Enum	Voice, Text, or Mixed
rawText	String	Transcribed or typed command text
timestamp	Timestamp	Capture time

Intent

Attribute	Type	Description
intentId	UUID	Normalized goal
name	String	Intent label
confidence	Float	Classifier confidence

Plan

Attribute	Type	Description
planId	UUID	Ordered set of actions
summary	String	One-line description
status	Enum	Proposed, Running, Succeeded, Failed

Action

Attribute	Type	Description
actionId	UUID	Atomic operation
toolId	UUID	Tool to invoke
parameters	JSON	Tool inputs
result	JSON	Tool outputs or return value
riskLevel	Enum	Low, Medium, or High

Tool

Attribute	Type	Description
toolId	UUID	Unique tool capability
name	String	Human-friendly tool name
description	String	Purpose and constraints
isDestructive	Boolean	Writes, installs, or deletes

AuditLog

Attribute	Type	Description
auditId	UUID	Append-only log entry
eventType	Enum	Example: CommandCaptured, ActionSucceeded
details	String/JSON	Minimal facts about the event
createdAt	Timestamp	When the entry was written

8 The Scope of the Product

This section defines the product boundary and summarizes the main ways in which users will interact with the system. The use cases in this section

describe the major user-visible capabilities that motivate the functional requirements in Section 9.

8.1 Product Boundary

Proxi is an assistive desktop orchestration layer that enables users to perform common computer tasks through natural speech or text. It translates user intent into safe, scoped actions and provides clear feedback before, during, and after execution.

Boundary Definition: The product coordinates supported actions across the existing desktop environment; it does not replace full third-party applications.

In-Scope Features:

- Multi-modal input capture and interpretation (voice and text).
- Desktop action execution through MCP-compatible tool integrations.
- Safety guardrails, including confirmation before risky or destructive operations.
- Accessible system feedback through text, speech, or both.
- Short-term session context and local audit logging for traceability.

Out-of-Scope Features:

- Full replacement of third-party tools such as complete email clients, browsers, or office suites.
- Unattended remote administration or unsupervised control of user systems.
- Enterprise-scale policy governance and organization-wide IT management.
- Default long-term cloud retention of user interaction data.

8.2 Product Use Case Table

Table 3 summarizes the main product use cases.

Table 3: Product Use Case Table

Use Case	Actors	Description	Preconditions	Outcome
PUC-01: Open Application	User	Open a named app.	App installed; automation allowed.	App launched or focused.
PUC-02: Open and Read File	User	Open a file; preview or read.	File exists; within scope.	File opened; content visible.
PUC-03: Browse and Search	User	Go to a URL or search.	Browser available; internet on.	Page or results shown.
PUC-04: Draft Message or Email	User	Create a short draft.	Mail or editor available.	Draft opens prefilled.
PUC-05: Save or Organize File	User	Save As, move, or rename.	Paths allowed; confirm overwrite.	Operation completed.
PUC-06: Interact with Open App	User	Perform simple in-app actions.	Target app open; allowed scope.	Action done or reason shown.

8.3 Individual Product Use Cases (PUC's)

This subsection expands each product use case with concise behavioral detail to support design alignment and test planning.

PUC-01: Open Application

Intent: Open or focus a named application quickly.

Behavior: Proxi resolves candidate matches, requests disambiguation if needed, validates execution scope, and opens or focuses the selected app.

Result: The requested application becomes available and a clear success message is returned.

PUC-02: Open and Read File

Intent: Access and review file contents with minimal manual navigation.

Behavior: Proxi resolves file identity within allowed scope, opens it in the default viewer, and can optionally provide readback or short summary assistance.

Result: The user can inspect target content and continue with follow-up tasks.

PUC-03: Browse and Search

Intent: Navigate to a destination or find information on the web.

Behavior: Proxi validates user input as URL or query, launches the browser path, and requests confirmation when destinations appear risky or ambiguous.

Result: The correct page or search results are displayed with clear status feedback.

PUC-04: Draft Message or Email

Intent: Produce communication drafts from natural-language instructions.

Behavior: Proxi extracts recipient and message details, opens the draft surface in the chosen application, and pre-fills content for user review.

Result: An editable draft is created, with final send control remaining with the user.

PUC-05: Save or Organize File

Intent: Complete common file organization tasks safely.

Behavior: Proxi validates paths and operation type, handles conflicts (for example name collisions), and requests confirmation for overwrite or destructive impact.

Result: Requested file-state changes are completed safely or a clear blocker is reported.

PUC-06: Interact with Open App

Intent: Perform focused actions inside an already-open application.

Behavior: Proxi executes simple in-app operations (select, click, type, toggle) and asks follow-up questions when intent is underspecified.

Result: The target action is completed or failure is explained with a suggested next step.

8.4 System State Diagram

The following state model captures the critical safety-oriented flow of a Proxi interaction. This visual specification complements the formal interaction protocol in Section 9.3.

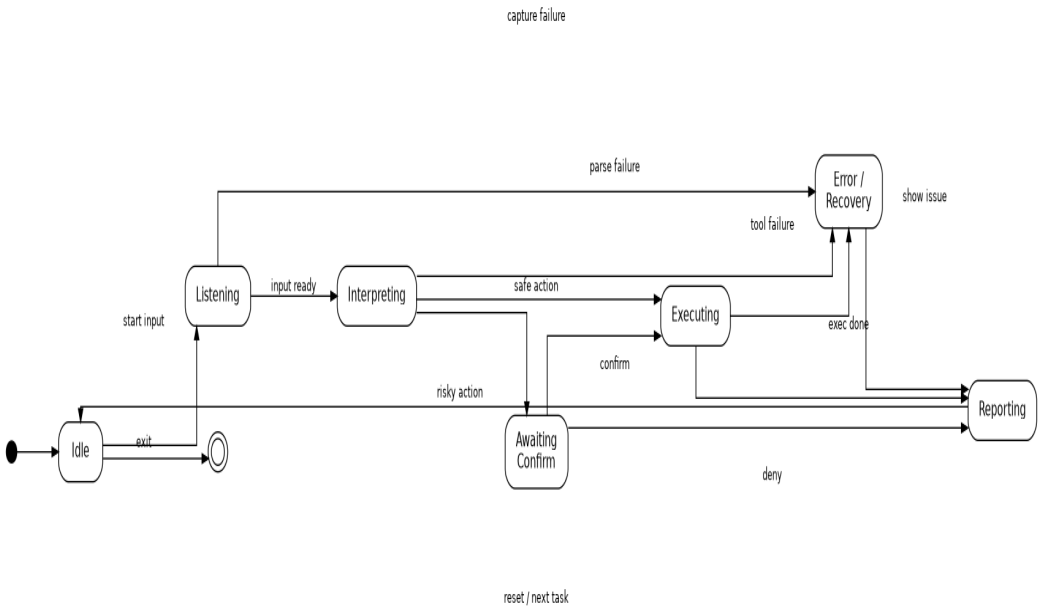

Figure 2: System State Diagram

9 Functional Requirements

This section states what the system does. To preserve traceability with the existing design and V&V documents, the requirement identifiers are kept the same even where the wording has been refined.

9.1 Functional Requirements

ID	**FUNC.R.1**
Requirement	The system shall accept user input through both speech and text interfaces.
Rationale	Proxi must support hands-free accessibility use and fallback text interaction for noisy or inaccessible environments.
Fit Criterion	Supported voice and text commands are accepted and passed into the processing pipeline during system testing.
Priority	High

ID	**FUNC.R.2**
Requirement	The system shall convert speech input into text using an automatic speech recognition module.
Rationale	Speech input must be normalized into text before intent interpretation.
Fit Criterion	Speech input is converted into text suitable for downstream intent processing.
Priority	High

ID	**FUNC.R.3**
Requirement	The system shall interpret user intent from natural language input to determine the desired task.
Rationale	The system must determine what task the user wants without requiring technical commands.
Fit Criterion	Natural-language requests are mapped to a structured intent for the task planner.
Priority	High

ID	**FUNC.R.4**
Requirement	The system shall create and execute a plan that invokes one or more tools on behalf of the user.
Rationale	User-visible task completion depends on safe composition of lower-level actions.
Fit Criterion	The system achieves at least `TASK_SUCCESS_MIN` successful completion on representative end-to-end task scenarios.
Priority	High

ID	**FUNC.R.5**
Requirement	The system shall present task status and task results to the user in text, speech, or both.
Rationale	Users need continuous feedback to understand what the system is doing.
Fit Criterion	Each executed command produces a user-visible status or result message.
Priority	High

ID	**FUNC.R.6**
Requirement	The system shall maintain short-term session context so that users can issue follow-up commands.
Rationale	Natural interaction requires short follow-up references such as "that" or "the last one."
Fit Criterion	Follow-up references to recent entities or actions are resolved using session context.
Priority	Medium

ID	**FUNC.R.7**
Requirement	The system shall log user interactions and task results for traceability and debugging.
Rationale	Traceability is needed for debugging, auditing, and future improvement.
Fit Criterion	Each user session produces timestamped log entries for commands, plans, and outcomes.
Priority	Medium

ID	**FUNC.R.8**
Requirement	The system shall allow users to complete supported tasks without requiring low-level system interfaces such as raw file paths or command-line tools.
Rationale	The product goal is high-level accessible control, not low-level system administration.
Fit Criterion	Users can complete supported tasks through the Proxi interface without needing raw OS commands or internal file-system knowledge.
Priority	High

ID	**FUNC.R.9**
Requirement	The system shall request explicit confirmation before executing privileged or destructive actions.
Rationale	Risky tasks must not execute silently.
Fit Criterion	Each privileged or destructive action triggers a confirmation step before execution.
Priority	High

9.2 Traceability Summary

Table 4 summarizes how the highest priority personas and goals map to the main requirement categories.

Table 4: Persona and Goal Traceability Matrix

Source	**Main Need**	**Primary Requirement Links**
P1 Amrita	Clear feedback, low complexity, readable interface	EOU.R.1–EOU.R.3, UAP.R.1–UAP.R.3, ACC.R.2, APP.1–APP.5
P2 Leo	Hands-free control, captions, accessible workflows	FUNC.R.1–FUNC.R.9, ACC.R.1–ACC.R.2, SAF.R.1–SAF.R.2
P3 Ari	Efficient repeated workflows and fast task completion	FUNC.R.4–FUNC.R.8, SAL.R.1–SAL.R.2, SOE.R.1–SOE.R.2
G-1	Accessibility	ACC.R.1–ACC.R.2, EOU.R.1–EOU.R.3, APP.1–APP.5
G-2	Independent task completion	FUNC.R.1–FUNC.R.9, POA.R.1–POA.R.2, SAL.R.1–SAL.R.2
G-3	Safety and trust	SAF.R.1–SAF.R.2, UAP.R.2–UAP.R.3, ROFT.R.1–ROFT.R.2

9.3 Formalization of Interaction Safety Protocol

A critical portion of the system is the user-command execution flow for potentially risky actions. Proxi shall follow the state machine below.

Let

$$S = \{Idle, Listening, Interpreting, AwaitingConfirm, Executing, Reporting\}.$$

Let

$$E = \{startInput, inputReady, intentReady, risky, confirm, deny, execDone, reset\}.$$

Let

$$\delta : S \times E \to S.$$

The required transitions are:

$$\delta(Idle, startInput) = Listening$$

$$\delta(Listening, inputReady) = Interpreting$$

$$\delta(Interpreting, intentReady) = Executing$$

$$\delta(Interpreting, risky) = AwaitingConfirm$$

$$\delta(AwaitingConfirm, confirm) = Executing$$

$$\delta(AwaitingConfirm, deny) = Reporting$$

$$\delta(Executing, execDone) = Reporting$$

$$\delta(Reporting, reset) = Idle$$

Safety invariant:

$$\forall a \in RiskyActions : Execute(a) \Rightarrow Confirmed(a)$$

This invariant states that any risky action may execute only after an explicit confirmation event has been observed.

10 Look and Feel Requirements

This section describes the intended visual style and presentation of the system. These requirements guide interface consistency and are supported by the usability and accessibility requirements that follow.

10.1 Appearance Requirements

ID	**APP.1**
Requirement	The interface shall look simple and familiar, like a normal desktop tool with a clear status area that shows when the system is listening, thinking, or ready.
Rationale	Users should understand the current system state at a glance.
Fit Criterion	The main interface visibly displays system state in all tested interaction modes.
Priority	High

ID	**APP.2**
Requirement	Main actions such as listen, stop, undo, and confirm shall be clearly visible as controls on the main screen.
Rationale	Critical controls must be discoverable without deep navigation.
Fit Criterion	All critical controls are accessible within one main view.
Priority	High

ID	**APP.3**
Requirement	The interface shall avoid clutter and show only the most important options by default.
Rationale	Minimalism reduces cognitive load.
Fit Criterion	Advanced settings are separated from the main interaction flow.
Priority	Medium

ID	**APP.4**
Requirement	Text and buttons shall be large enough to be easy to read and select.
Rationale	Large text and controls support elderly and low-vision users.
Fit Criterion	The interface remains usable at 200% text scaling.
Priority	High

ID	**APP.5**
Requirement	A light and dark theme option shall be available.
Rationale	Users have different comfort and visibility preferences.
Fit Criterion	Users can switch between light and dark themes without loss of functionality.
Priority	Low

10.2 Style Requirements

ID	**STY.1**
Requirement	Labels and messages shall use short, plain language with no unnecessary technical jargon.
Rationale	The target users should not need specialist knowledge to understand system feedback.
Fit Criterion	Reviewed messages avoid internal or tool-specific jargon in user-facing flows.
Priority	High

ID	**STY.2**
Requirement	When the system speaks, the same content shall also appear on screen as captions.
Rationale	Captions improve accessibility and reinforce spoken feedback.
Fit Criterion	All spoken responses have matching on-screen text.
Priority	High

ID	**STY.3**
Requirement	Icons shall be consistent and paired with short text labels.
Rationale	Text labels reduce ambiguity in icon interpretation.
Fit Criterion	Primary icons used in the interface have corresponding labels.
Priority	Medium

11 Usability and Humanity Requirements

This section captures how approachable, learnable, understandable, and accessible the system should be for the target users identified earlier.

11.1 Ease of Use Requirements

ID	**EOU.R.1**
Requirement	The system shall be easy for new users to start using.
Rationale	Low-friction onboarding is necessary for accessibility-focused users.
Fit Criterion	Basic tasks such as opening an application or reading a file require only one simple command or button press in typical flows.
Priority	High

ID	**EOU.R.2**
Requirement	The system shall have an intuitive interface that does not overwhelm users.
Rationale	Users should not struggle to find core actions.
Fit Criterion	Main actions such as listen, stop, undo, and confirm are clearly visible and easy to access.
Priority	High

ID	**EOU.R.3**
Requirement	After a short introduction, users should be able to complete most everyday tasks without assistance.
Rationale	The system should become useful quickly in real use.
Fit Criterion	Task effectiveness improves by at least `USABILITY_GAIN_MIN` over a baseline workflow.
Priority	High

11.2 Personalization and Internationalization Requirements

ID	**PER.R.1**
Requirement	The system shall adapt to a user's speech patterns over time to improve recognition quality.
Rationale	Repeated use should reduce friction for frequent users.
Fit Criterion	The system stores and applies allowed local personalization settings.
Priority	Medium

ID	**PER.R.2**
Requirement	The system shall support Canadian English standards for language, date, and time formatting and allow future localization if needed.
Rationale	The project is developed in a Canadian academic context.
Fit Criterion	Default language and formatting reflect Canadian English conventions.
Priority	Medium

11.3 Learning Requirements

ID	**LEA.R.1**
Requirement	The system shall have a short learning curve, allowing new users to understand and use basic commands quickly.
Rationale	The first-use experience should not require formal training.
Fit Criterion	New users can understand the basic interaction model within a short guided introduction.
Priority	High

ID	**LEA.R.2**
Requirement	The system shall provide a list of example commands or a "What can I say?" help option.
Rationale	Users need discoverable examples to build confidence.
Fit Criterion	Example commands are accessible through the interface during use.
Priority	High

11.4 Understandability and Politeness Requirements

ID	**UAP.R.1**
Requirement	The system shall use simple and clear language for all user-facing messages.
Rationale	Clarity reduces user confusion and error recovery time.
Fit Criterion	User-facing messages avoid unexplained technical terminology.
Priority	High

ID	**UAP.R.2**
Requirement	The system shall confirm actions that could change or delete important files or settings before executing them.
Rationale	Clear confirmation protects user trust and data.
Fit Criterion	All destructive or privileged actions trigger explicit confirmation.
Priority	High

ID	**UAP.R.3**
Requirement	Error messages shall be polite, explain the issue in plain language, and suggest a next step.
Rationale	Helpful error recovery is part of usable accessibility support.
Fit Criterion	Tested error messages include cause and suggested recovery step.
Priority	Medium

11.5 Accessibility Requirements

ID	**ACC.R.1**
Requirement	The system shall allow all core actions, including setup and exit, to be performed using voice commands without requiring a keyboard or mouse.
Rationale	Hands-free control is a core accessibility goal.
Fit Criterion	Core actions are completable through the voice-only path during testing.
Priority	High

ID	**ACC.R.2**
Requirement	The system shall comply with recognized accessibility standards such as WCAG 2.2 and AODA where applicable to the interface.
Rationale	The system should follow established accessibility guidance rather than ad hoc design choices.
Fit Criterion	The interface provides captions for spoken responses, supports text scaling, and ensures sufficient contrast in tested views.
Priority	High

12 Performance Requirements

This section defines quantitative expectations for latency, accuracy, fault tolerance, and capacity. Where possible, the fit criteria are expressed using the symbolic constants from Table 1.

12.1 Speed and Latency Requirements

ID	**SAL.R.1**
Requirement	The system shall begin responding to a voice command within `QUIET_RESP_MAX` under normal quiet conditions.
Rationale	Fast acknowledgment improves confidence and usability.
Fit Criterion	Measured response-start time does not exceed `QUIET_RESP_MAX` in quiet-room testing.
Priority	High

ID	**SAL.R.2**
Requirement	Common local actions such as opening an application or navigating files shall complete within `TASK_TIME_MAX` on standard hardware.
Rationale	Core task execution must feel responsive.
Fit Criterion	Representative local actions complete within `TASK_TIME_MAX` during testing.
Priority	High

12.2 Safety-Critical Requirements

ID	**SAF.R.1**
Requirement	Any action that changes files or system settings shall require user confirmation before execution.
Rationale	Potentially harmful actions must not run silently.
Fit Criterion	All tested risky actions require explicit confirmation.
Priority	High

ID	**SAF.R.2**
Requirement	The system shall allow users to undo or roll back critical actions when such rollback is technically feasible.
Rationale	Undo support reduces the impact of mistakes.
Fit Criterion	Supported critical actions provide an undo or rollback path.
Priority	Medium

12.3 Precision or Accuracy Requirements

ID	**POA.R.1**
Requirement	The system shall correctly recognize at least `QUIET_ACC_MIN` of supported commands in a quiet environment.
Rationale	Quiet-condition recognition defines the baseline usability target.
Fit Criterion	Measured recognition accuracy in quiet-room testing is at least `QUIET_ACC_MIN`.
Priority	High

ID	**POA.R.2**
Requirement	In a moderately noisy indoor environment, the system shall maintain at least `MOD_NOISE_ACC_MIN` command recognition accuracy.
Rationale	The system must remain usable outside ideal test conditions.
Fit Criterion	Measured recognition accuracy in moderate-noise testing is at least `MOD_NOISE_ACC_MIN`.
Priority	Medium

12.4 Robustness or Fault-Tolerance Requirements

ID	**ROFT.R.1**
Requirement	The system shall log and report input or execution errors without crashing or losing user data.
Rationale	Robust recovery is necessary for trust and debugging.
Fit Criterion	Invalid or failed tasks generate visible feedback and log entries without terminating the application.
Priority	High

ID	**ROFT.R.2**
Requirement	The system shall continue running even if it receives an invalid or incomplete command.
Rationale	One bad input must not collapse the whole application.
Fit Criterion	Invalid commands are rejected gracefully and the application remains operational.
Priority	High

12.5 Capacity Requirements

ID	**CAP.R.1**
Requirement	The system shall support continuous operation for at least `SESS_DURATION_MIN` without significant degradation.
Rationale	The product should be suitable for extended daily use.
Fit Criterion	The application remains usable for at least `SESS_DURATION_MIN` in continuous testing.
Priority	Medium

ID	**CAP.R.2**
Requirement	CPU usage shall remain under `APP_CPU_MAX` during normal operation on standard hardware.
Rationale	The application should not excessively burden the host system.
Fit Criterion	Measured CPU usage remains below `APP_CPU_MAX` during normal-use scenarios.
Priority	Medium

12.6 Scalability or Extensibility Requirements

ID	**SOE.R.1**
Requirement	The system shall allow new features or modules to be added without major changes to existing functionality.
Rationale	The product should remain evolvable after the capstone period.
Fit Criterion	New tools can be added through the modular architecture with limited changes to existing components.
Priority	Medium

ID	**SOE.R.2**
Requirement	Any new feature or skill shall load and become available for use within `APP_LOAD_TIME` after initialization of the interface.
Rationale	Dynamic features should not significantly slow user interaction.
Fit Criterion	Added features become available within `APP_LOAD_TIME` in supported workflows.
Priority	Low

12.7 Longevity Requirements

ID	**LON.R.1**
Requirement	The system shall retain user settings and allowed personalization data after updates.
Rationale	Users should not lose preferred settings after upgrades.
Fit Criterion	Existing local settings persist across normal update scenarios.
Priority	Medium

ID	**LON.R.2**
Requirement	The system shall allow rollback to a previous version within `ROLLBACK_MAX` if an update fails.
Rationale	Recovery from failed updates is necessary for maintainability.
Fit Criterion	A failed update can be reversed within `ROLLBACK_MAX`.
Priority	Low

13 Operational and Environmental Requirements

This section describes the environments in which the system is expected to operate and the interfaces it must support.

13.1 Expected Physical Environment

ID	**EPE.R.1**
Requirement	The system shall operate on personal computing devices such as laptops and desktop computers with standard hardware including a microphone, speakers, and a stable internet connection when needed.
Rationale	The target environment is ordinary consumer desktop hardware.
Fit Criterion	Core Proxi workflows execute on supported consumer laptops and desktops.
Priority	High

ID	**EPE.R.2**
Requirement	The system shall be designed to work optimally for indoor environments where noise levels are moderate, up to `INDOOR_NOISE_MAX`.
Rationale	Proxi is primarily intended for indoor personal-computing contexts.
Fit Criterion	Voice interaction remains functional in moderate indoor noise.
Priority	Medium

13.2 Wider Environment Requirements

ID	**WER.R.1**
Requirement	The system shall integrate safely within the desktop ecosystem, ensuring that its automation does not interfere with unrelated running applications.
Rationale	Automation must be scoped to the intended task.
Fit Criterion	Tested actions affect only the targeted application or resource.
Priority	High

13.3 Requirements for Interfacing with Adjacent Systems

ID	**IAS.R.1**
Requirement	The system shall provide safe interfaces for integration with third-party tools and applications such as email tools, browsers, and file systems.
Rationale	Proxi exists to orchestrate adjacent tools rather than replace them.
Fit Criterion	Supported integrations are invokable through controlled interfaces.
Priority	High

ID	**IAS.R.2**
Requirement	The system shall support standard communication mechanisms such as HTTPS and MCP where appropriate.
Rationale	Standard protocols simplify interoperability.
Fit Criterion	External integrations use supported standard communication methods.
Priority	Medium

13.4 Productization Requirements

ID	**PRD.R.1**
Requirement	The system shall be packaged as a deployable desktop or desktop-style application with minimal setup steps.
Rationale	Users should be able to install and run the product easily.
Fit Criterion	A documented installation path exists and is usable by testers.
Priority	High

ID	**PRD.R.2**
Requirement	The product shall include installation documentation and user onboarding instructions in the repository.
Rationale	Basic support material is necessary for handoff and evaluation.
Fit Criterion	Installation and onboarding instructions are present in the repository.
Priority	High

ID	**PRD.R.3**
Requirement	The design shall allow modular upgrades, enabling future integration of improved AI models or additional tools.
Rationale	The product should remain maintainable beyond the capstone period.
Fit Criterion	The architecture isolates tool and model integrations behind defined interfaces.
Priority	Medium

13.5 Release Requirements

ID	**REL.R.1**
Requirement	Releases shall follow a documented versioning scheme.
Rationale	Clear versioning supports reproducibility and maintenance.
Fit Criterion	Released builds are labeled using a consistent scheme.
Priority	Medium

ID	**REL.R.2**
Requirement	The release package shall be version-controlled and archived for reproducibility.
Rationale	Capstone evaluation and later maintenance require reproducible builds.
Fit Criterion	Released versions are preserved in version control with tags or equivalent records.
Priority	Medium

14 Maintainability and Support Requirements

This section describes the qualities needed to keep the product modifiable, supportable, and adaptable over time.

14.1 Maintenance Requirements

ID	**MTN.R.1**
Requirement	The project shall include a short README and setup steps so new contributors can build and run it without help.
Rationale	Low friction onboarding improves maintainability.
Fit Criterion	A new contributor can build and run the project using repository documentation.
Priority	High

ID	**MTN.R.2**
Requirement	Adding a new MCP tool shall require only a new module and a registry entry, without modifying unrelated tools.
Rationale	Tool extensibility should not create broad regressions.
Fit Criterion	A new tool can be integrated by localized changes.
Priority	Medium

14.2 Supportability Requirements

ID	**SUP.R.1**
Requirement	The release shall include a simple user guide with a quick start, common commands, and basic troubleshooting.
Rationale	Users need lightweight self-help material.
Fit Criterion	The delivered documentation includes quick start, common commands, and troubleshooting content.
Priority	High

ID	**SUP.R.2**
Requirement	The application shall offer a report-problem workflow that bundles recent logs and basic system information with the user's consent.
Rationale	Support data helps diagnose failures without silent data collection.
Fit Criterion	Users can opt in to produce a support bundle from the interface.
Priority	Medium

14.3 Adaptability Requirements

ID	**ADP.R.1**
Requirement	The system shall run on current supported versions of Windows and macOS on standard hardware.
Rationale	Users do not all share the same desktop environment.
Fit Criterion	Core functionality is demonstrated on supported Windows and macOS systems.
Priority	High

ID	**ADP.R.2**
Requirement	The system shall continue to function if the user switches default applications by updating configuration only.
Rationale	The system should adapt to ordinary desktop configuration changes.
Fit Criterion	Default application changes do not require code changes to restore supported workflows.
Priority	Medium

15 Security Requirements

This section defines the access-control, integrity, privacy, audit, and immunity requirements needed to operate Proxi safely.

15.1 Access Requirements

ID	**ACS.R.1**
Requirement	The application shall not require a Proxi-managed user registration or login to operate.
Rationale	Authentication should not become a barrier to accessibility.
Fit Criterion	Users can access core functionality without Proxi-managed credentials.
Priority	High

ID	**ACS.R.2**
Requirement	Any necessary permissions, such as microphone or automation access, shall be explicitly granted by the user.
Rationale	Permission use must be visible and consent-based.
Fit Criterion	Permission-dependent actions do not proceed without the required grant.
Priority	High

15.2 Integrity Requirements

ID	**INT.R.1**
Requirement	The system shall protect stored or transmitted data from unauthorized alteration or corruption.
Rationale	Users must be able to trust task results and system records.
Fit Criterion	Tested interactions preserve the integrity of local logs and task data.
Priority	High

15.3 Privacy Requirements

ID	**PRIV.R.1**
Requirement	Any stored user data shall be handled in a manner consistent with PIPEDA.
Rationale	The project operates in a Canadian context and should respect Canadian privacy expectations.
Fit Criterion	Stored data handling follows documented privacy constraints and minimal retention.
Priority	High

ID	**PRIV.R.2**
Requirement	If external APIs are used, only the minimum data necessary to complete the request may be transmitted, and the user shall be informed when third-party services are involved.
Rationale	Users should understand when their request leaves the local device.
Fit Criterion	External-service workflows disclose third-party involvement and use minimal task-relevant data.
Priority	High

15.4 Audit Requirements

ID	**AUD.R.1**
Requirement	The system shall maintain local audit records of significant user commands, confirmations, and action outcomes.
Rationale	Operational traceability is needed for debugging and accountability.
Fit Criterion	Significant actions produce local audit entries during testing.
Priority	Medium

ID	**AUD.R.2**
Requirement	Audit records shall avoid storing unnecessary sensitive content.
Rationale	Traceability should not require excessive privacy cost.
Fit Criterion	Audit entries focus on operational facts rather than unnecessary content payloads.
Priority	Medium

15.5 Immunity Requirements

ID	**IMM.R.1**
Requirement	The application shall reduce exposure to denial-of-service, injection, and unauthorized-access risks.
Rationale	Even a local desktop assistant can introduce abuse surfaces.
Fit Criterion	The design uses scoped permissions, input validation, and restricted tool invocation paths.
Priority	High

ID	**IMM.R.2**
Requirement	Third-party libraries and dependencies shall be kept reasonably up to date and checked for known vulnerabilities.
Rationale	Dependency hygiene is part of secure maintenance.
Fit Criterion	Dependencies are reviewed and updated as part of the release workflow.
Priority	Medium

16 Cultural Requirements

This section captures cultural neutrality and inclusiveness requirements for the interface and user-facing content.

16.1 Cultural Requirements

ID	**CULR.R.1**
Requirement	The application shall use polite, culturally neutral language and avoid slang or references that may confuse or offend users.
Rationale	The product should remain broadly understandable and respectful.
Fit Criterion	Reviewed user-facing text avoids slang and culturally narrow references.
Priority	Low

ID	**CULR.R.2**
Requirement	The application shall be designed so that future language expansion is possible without major redesign.
Rationale	The product may eventually serve a wider linguistic audience.
Fit Criterion	User-facing strings are separable from core execution logic.
Priority	Low

17 Compliance Requirements

This section defines legal and standards-related expectations relevant to the product and its environment.

17.1 Legal Requirements

ID	**LGL.R.1**
Requirement	The application shall adhere to PIPEDA for any stored user data.
Rationale	Canadian privacy law is the most relevant legal baseline here.
Fit Criterion	Stored-data handling is documented in a way consistent with PIPEDA constraints.
Priority	High

ID	**LGL.R.2**
Requirement	The application shall be distributed under the MIT license or another approved project license.
Rationale	A clear distribution license is needed for handoff and reuse.
Fit Criterion	The repository contains the selected project license.
Priority	Medium

17.2 Standards Compliance Requirements

ID	**STDCOMP.R.1**
Requirement	The application shall follow WCAG 2.2 guidance where applicable to the user interface.
Rationale	Accessibility should be grounded in recognized standards.
Fit Criterion	Tested interface views satisfy targeted WCAG checks used in V&V.
Priority	High

ID	**STDCOMP.R.2**
Requirement	The application should align with relevant software-quality and security guidance such as ISO 25010 and ISO 27001 where practical in a capstone setting.
Rationale	These standards provide useful quality and security reference points.
Fit Criterion	The design and V&V artifacts reflect relevant quality and security attributes.
Priority	Low

18 Traceability from Requirements to Use Cases

This section maps the requirements to the business and product use cases so that each major requirement can be traced to a user-visible interaction or workflow.

Table 5: Traceability from Requirements to Use Cases

Requirement	**Use Case(s)**
FUNC.R.1	BUC-01, PUC-01, PUC-02, PUC-03, PUC-04, PUC-05, PUC-06
FUNC.R.2	BUC-01, PUC-01, PUC-02, PUC-03, PUC-04, PUC-05, PUC-06
FUNC.R.3	BUC-01, PUC-01, PUC-02, PUC-03, PUC-04, PUC-05, PUC-06
FUNC.R.4	BUC-01, PUC-01, PUC-02, PUC-03, PUC-04, PUC-05, PUC-06
FUNC.R.5	All PUCs
FUNC.R.6	PUC-02, PUC-03, PUC-04, PUC-05, PUC-06
FUNC.R.7	All PUCs
FUNC.R.8	All PUCs
FUNC.R.9	PUC-03, PUC-04, PUC-05, PUC-06
ACC.R.1	All PUCs
ACC.R.2	All PUCs
SAF.R.1	PUC-03, PUC-04, PUC-05, PUC-06
SAF.R.2	PUC-04, PUC-05, PUC-06
SAL.R.1	All voice-driven PUCs
SAL.R.2	PUC-01, PUC-02, PUC-03, PUC-05, PUC-06
POA.R.1	All voice-driven PUCs
POA.R.2	All voice-driven PUCs
ROFT.R.1	All PUCs
ROFT.R.2	All PUCs
AUD.R.1	All PUCs
PRIV.R.2	PUC-03, PUC-04, PUC-06

19 Open Issues

This section outlines unresolved questions and challenges that may impact the system and require more validation.

- **OI.1** MCP-based system-level control is still being hardened. Additional testing is needed to verify reliable and safe command execution across different application types.
- **OI.2** Cross-platform consistency is not yet fully validated. Behavior and performance across Windows, macOS, and Linux require broader test coverage.
- **OI.3** Prevention of unintended or unauthorized actions is still under investigation. Additional permission checks, confirmation policies, and safety constraints are needed.

20 Off-the-Shelf Solutions

This section summarizes existing technologies and design patterns that can be reused instead of being built from scratch.

20.1 Ready-Made Products

Several existing tools can cover core parts of Proxi without reimplementation:

- **Speech-to-Text (STT):** STT engines capture microphone input and return text suitable for intent processing. Built-in OS speech-recognition capabilities may be used when they are available and appropriate.
- **Text-to-Speech (TTS):** TTS services can synthesize spoken responses for accessibility-oriented feedback (for example, GPT-4o mini TTS).
- **Desktop Automation:** OS-level automation libraries can open applications, switch windows, and perform scoped UI actions.
- **Language Understanding:** LLM APIs can support command interpretation, decomposition, and plan generation (for example, GPT-4o mini as a primary planning model in baseline scope).

- **Storage and Logging:** Lightweight local storage (for example SQLite or structured log files) can persist settings and audit records.

20.2 Reusable Components

Components that can be reused or lightly adapted include:

- **MCP tool skeletons:** Shared templates and registry conventions that make new tool integration predictable.
- **OS adapters:** Platform-specific adapters that expose a stable interface for common actions (open app, focus window, and file operations).
- **Prompt templates:** Reusable prompt patterns for common tasks such as opening apps or files, browse/search workflows, and draft-generation actions.

20.3 Products That Can Be Copied

Proven interaction patterns that can be adopted include:

- **Command palette pattern:** A unified search surface for actions and recent targets.
- **Clear confirmation modal:** A concise approval step for destructive or high-risk actions.
- **First-run permission checklist:** Guided permission setup for microphone, automation, and app access. When additional app access is needed, permission is requested again before further command execution.

21 New Problems

This section documents possible problems introduced by the new product so they can be considered during design, testing, and deployment.

21.1 Effects on the Current Environment

Potential effects include:

- Users may become dependent on natural-language control.
- Additional CPU and memory usage may affect other applications under constrained hardware conditions.
- Existing workflows may change when users adopt Proxi for repeated tasks.
- AI-driven automation may produce unexpected interactions with other software if sandboxing and scope controls are insufficient.

21.2 Effects on the Installed Systems

Potential effects include:

- Some applications may require additional permissions or interface allowances.
- OS or application updates may temporarily disrupt automation.
- Extra runtime dependencies may need to be installed.
- Increased network usage for cloud-backed capabilities may affect other applications that share connectivity.

21.3 Potential User Problems

Potential user problems include:

- Misinterpretation of voice commands that leads to incorrect task execution.
- Distrust when the system behaves unexpectedly.
- Confusion caused by too much feedback or too many prompts.
- Reduced recognition accuracy for atypical speech patterns.

21.4 Limitations in the Anticipated Implementation Environment That May Inhibit the New Product

Potential limitations include:

- Limited or unstable internet connectivity for cloud-dependent services.
- Low-end hardware causing slow execution.
- Restrictive applications that do not expose controllable interfaces.

21.5 Follow-Up Problems

Potential follow-up problems include:

- Ongoing maintenance will be required to remain compatible with updated applications and OS versions.
- Privacy and security risks must be monitored as integrations change over time.

22 Tasks

This section explains how the project will be carried out and how requirement priorities influence scheduling and development order.

22.1 Project Planning

The project will follow an iterative and incremental development approach. Early prototypes and continuous feedback will be used to address defects and scope issues early. Core stages include requirements, design, implementation, testing, and deployment.

Developers are also responsible for preparing and maintaining unit and functional test cases throughout the lifecycle.

22.2 Planning of the Development Phases

Requirement priority affects scheduling as follows: accessibility, safety, and core task execution are implemented first; usability and reliability improvements follow; advanced extensibility and stretch features are scheduled later. Although phase names are listed below, execution is iterative rather than strictly linear.

1. **Requirements Phase:** Define system capabilities, constraints, and stakeholder-validated requirements; gather input from target users and establish both functional and non-functional requirement baselines.
2. **Design Phase:** Establish architecture, component boundaries, technology choices, and integration patterns; select off-the-shelf models and technologies; produce supporting diagrams and data-flow artifacts.
3. **Development Phase:** Implement prioritized features, integrate tools, and update tests for each iteration; include both automated and manual test-case development for new or changed functionality.
4. **Testing and Validation Phase:** Execute functional, usability, accessibility, and regression testing across target environments, including representative users and low-spec hardware profiles.
5. **Deployment and Handoff Phase:** Prepare release artifacts, update user documentation, and support transition to maintainers; ensure installation and onboarding remain simple and version history remains traceable.

23 Migration to the New Product

This section explains what is needed for users to transition from their current workflow to the new product.

23.1 Requirements for Migration to the New Product

There are no special migration steps beyond installing Proxi and granting the usual permissions on first run, such as microphone and automation access.

For access to specific applications and folders, Proxi requests permission when needed.

Existing workflows and applications do not need replacement; users only need to become familiar with the voice/text command interaction model.

23.2 Data That Has to be Modified or Translated for the New System

No data conversion is needed. The tool works with the user's existing files and applications. The only new data created is local configuration and local logs if enabled; no prior-tool data import or translation is required.

24 Costs

This section summarizes the project cost perspective.

The project will aim to use open-source and off-the-shelf solutions whenever possible. Since this project is a student project as part of a course, development labour costs are not taken into account. The primary costs are related to third-party APIs, hosted services, or software licenses used during development and testing.

25 User Documentation and Training

This section describes the documentation and training material needed to support installation, onboarding, and everyday use.

25.1 User Documentation Requirements

- The system shall include a user manual explaining installation, setup, and configuration procedures.
- The documentation shall provide step-by-step instructions for performing common tasks via speech and text commands.

- The manual shall include troubleshooting guidance for common issues such as misinterpreted commands, failed task execution, or connectivity problems.
- The documentation shall provide an explanation of accessibility features, including options for speech rate, font size, and interface contrast.
- The system shall include inline help or tooltips within the interface for immediate guidance during use.
- Documentation shall be available in multiple formats, including PDF, HTML, and context-sensitive help within the software.

25.2 Training Requirements

There isn't a dedicated training program for the system but the user will be provided with example scenarios that they can follow to build intuition on how to direct the agentic system.

- Users shall be able to complete an introductory tutorial that demonstrates how to issue commands and interact with the AI agents.
- The system shall provide interactive training sessions or guided exercises to familiarize users with core functionalities such as sending emails, managing files, and performing web searches.
- Training materials shall include examples of natural language commands, including variations in phrasing to account for diverse user speech patterns.

26 Waiting Room

These are ideas that are being kept aside for now because they are not the most urgent:

- Offline mode using a small local model.
- Third-party app plugins.

- Multi-device task continuity (for example, start on one device and continue on another).
- Custom TTS voice options (deferred in baseline scope).
- Broader language support.

27 Ideas for Solution

Proxi should stay simple and safe. The main idea is a small desktop app that listens for a request, confirms anything risky in plain language, and then runs a scoped action through a set of MCP tools.

- **Local MCP server exposing scoped tools:** Tool actions run with low privilege and operate only within user-approved scopes.
- **Lightweight UI with status and captions:** The interface presents clear state indicators and accessible control points (for example listen, stop, undo, and confirm).
- **Voice in and text out with captions:** STT converts microphone input to text, and TTS can provide spoken output while captions remain visible.
- **Simple intent handling:** Common intents use reusable templates and concise follow-up questions when user requests are ambiguous.
- **Safety by default:** Destructive actions require explicit confirmation and permission scope checks.
- **Traceability:** A local audit log stores operational events while avoiding unnecessary sensitive content.

Appendix - Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?
4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.
5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.
6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

28 Reflections

28.1 Amanbeer Minhas Reflection

1. **What went well while writing this deliverable?**
   One thing that went well while writing this deliverable was that I had a much clearer understanding of what was expected compared to our previous submissions. The Volere template helped a lot because it gave a clear structure and description of what needed to be included in each section, so I was able to focus on writing instead of figuring out the format. As a team, we also communicated better and stayed consistent in how we wrote different sections, which made the final document more cohesive.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**
   The biggest challenge we faced was dividing up the work evenly across the team. At first, some team members started earlier and took on more tasks, while others joined later, leading to an imbalance in workload. This caused some sections to be rushed near the end. To resolve this, we agreed to plan task assignments ahead of time for future deliverables so that everyone has clear responsibilities and the work is distributed more fairly. This will help us stay more organized and avoid last-minute issues.

3. **How many of your requirements were inspired by speaking to your client(s) or their proxies?**
   It is hard to put an exact number on how many requirements were inspired by client conversations, but several key ones were shaped by those discussions. Our focus on accessibility and ease of use came directly from client feedback about making the system usable for people with motor or visual impairments. Additionally, requirements around safety, such as confirming system-level actions before execution, were influenced by stakeholder input about avoiding accidental or harmful commands.

4. **Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project?**
   Several courses I have taken will directly support our work on this project. Software Requirements (SFWRENG 3RA3) taught me how to gather and define clear requirements, which is essential for this deliverable. Software

Architecture and Design courses will help with structuring the system and designing modular components. Human-Computer Interfaces will be useful for designing an accessible and user-friendly interface. Finally, our programming and systems courses like Object-Oriented Programming and Concurrent Systems Design provide the technical foundation for building and connecting different parts of the system.

5. **What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project?**
   As a team, we will need to gain deeper knowledge of how to integrate the MCP (Model Context Protocol) effectively to control system-level actions. We also need to improve our understanding of operating system differences to make sure the solution works across Windows, macOS, and Linux. On the skills side, we will need to strengthen our testing practices, improve our technical writing for documentation, and enhance our team coordination and time management skills. Individually, I want to focus on improving my testing and integration skills, as that will be important for the reliability of our system.

6. **For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Which will each team member pursue, and why did they make this choice?**
   To improve our knowledge of MCP and system-level integration, we can study official documentation and build small prototype projects to experiment with its capabilities. We can also seek help from online forums and developer communities to learn from real-world use cases. For cross-platform knowledge, we can test our software on multiple operating systems throughout development and review OS-specific guidelines. To improve testing skills, we can practice writing unit and integration tests for smaller modules early in the project and use tools like automated testing frameworks. For writing and documentation, we plan to review examples of high-quality documentation and get feedback from instructors and TAs. I will personally focus on building prototypes to learn MCP integration and writing more automated tests, as these are most relevant to the parts of the project I am contributing to.

28.2 Gourob Podder Reflection

1. **What went well while writing this deliverable?**
   While writing this deliverable, we were able to clearly define the functional and operational requirements of our AI-powered assistive technology platform. Structuring requirements as black-box and verifiable statements ensured clarity and made it easier to anticipate testing criteria. Breaking down the system into modular components such as input handling, agent planning, and feedback mechanisms allowed us to comprehensively cover potential user interactions and system behaviors. Additionally, using a structured LaTeX format helped maintain consistency across sections.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**
   A key challenge was ensuring that all requirements were verifiable and free of design decisions. Initially, some requirements were too abstract (e.g., "the system shall be user-friendly"), which was not testable. We resolved this by reframing each requirement to include measurable criteria, such as task completion rates, recognition accuracy, or response times. Another pain point was anticipating all potential user problems and environmental constraints without making assumptions. We addressed this by reviewing accessibility guidelines, examining similar assistive technologies, and considering diverse user scenarios, including elderly users and people with disabilities.

3. **How many of your requirements were inspired by speaking to your client(s) or their proxies?**
   We don't have an exact measure of client impact, but the core requirements were either constructed or revised based on talking to potential clients. Discussions with accessibility-focused community members provided insight into what tasks are most important, potential user errors, and expectations for feedback mechanisms. This input was crucial for shaping functional requirements such as conversational memory and clarification prompt features.

4. **Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project?**
   Several courses will directly support our work on this project:

   - **Human-Computer Interaction (4HC3):** Understanding usability

principles and accessibility guidelines.

- **Software Engineering Requirements (3RA3):** Writing SRS, functional and non-functional requirements, and system design.
- **Operating Systems (3SH3):** Managing agent execution, concurrency, and integration with host systems.
- **Databases and Networking (4C03):** Handling logging, cloud-based APIs, and data persistence for the system.

5. **What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project?**
   To successfully complete this project, the team will need:

   - **Natural Language Processing (NLP) knowledge:** Understanding speech-to-text, intent recognition, and dialogue management.
   - **Agent-based system design:** Planning, execution, and tool integration for autonomous agents.
   - **Accessibility and usability skills:** Designing for diverse users, including those with disabilities or low technical proficiency.

6. **For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Which will each team member pursue, and why did they make this choice?**
   To acquire these skills:

   - **NLP knowledge:**
     (a) Complete online tutorials and courses on speech recognition and intent detection.
     (b) Implement small prototypes of text and speech processing modules for hands-on experience.
   - **Agent-based system design:**
     (a) Study academic papers and open-source projects on multi-agent systems.
     (b) Develop proof-of-concept agents in a controlled environment to understand task planning and execution.

- **Accessibility and usability skills:**
  (a) Review accessibility guidelines such as WCAG 2.1 and analyze existing assistive software.
  (b) Conduct user testing sessions with peers or volunteers to identify usability issues.

Each team member will focus on specific areas while gaining a breadth-first understanding of other areas. In my case, I have prior knowledge with NLP and basic agentic systems, so I will focus on reviewing accessibility studies related to software design.

28.3 Ajay Grewal Reflection

1. **What went well while writing this deliverable?**
   We agreed early on what Proxi should do, which is to turn prompts from a user into safe actions. That made the writing easier and overall requirements easier to understand and write down. This resulted in the sections coming together quickly. We also had good team communication, with everyone contributing ideas.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**
   We kept slipping into how this would be built instead of what Proxi would do. We fixed this by asking ourselves if this can be ideated properly without going too deep into the technology side. We also had many LaTeX issues when writing this regarding formatting. This was resolved through heavy trial and error.

3. **How many of your requirements were inspired by speaking to your client(s) or their proxies?**
   It is difficult to put an exact number on this, but many important ones were from those discussions. Making Proxi accessible and simple came from obvious client needs. Safety requirements, such as approving system level actions, were also heavily influenced by stakeholders.

4. **Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project?**
   Data structures and algorithms will help us in having efficient code and

data processing. Software requirements will help in defining structured requirements. Human computer interactions will help in making an effective and simple to use UI design for Proxi.

5. **What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project?**
   Better understanding of MCP tools, how to do OS automation, and how to build reliable tests/logs for our software. Personally, I really want to get better at MCP and OS automation as this will define how well Proxi works. We will also need to get better at writing tests and logs to ensure Proxi is reliable and safe. We also need to better work as a group to ensure the code work is separated well and everyone is contributing equally.

6. **For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Which will each team member pursue, and why did they make this choice?**
   Firstly, for this to work, a lot of trial and error will need to happen. Multiple prototypes will have to be built to see what works best, furthering our understanding of how MCP, and OS automation works, as well as writing effective test cases. I and each team member will also watch and read tutorials on MCP and OS automation to further understanding of these topics. This way we can all equally contribute to the technical design of Proxi.

28.4 Savinay Chhabra Reflection

1. **What went well while writing this deliverable?**
   The templates helped a lot as these explained what was expected of us. The weekly team meetings also helped us with coordinate with other team members for inspiration and ideas.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**
   Dividing the work proved challenging once again. The previous deliverable also suffered from an imbalanced workload. The problem is that we don't know the time commitment each section requires until we actually do that section. We split out the workload based on what we thougth

were our individual strengths however sections are not necessarily even in their workload. Another challenge was staying solution neutral while coming up with requirements.

3. **How many of your requirements were inspired by speaking to your client(s) or their proxies?**
   I don't recall excatly how many requirements were inspired by the clients, but the core requirements were made after understanding the needs of older adults. Other requirements were inferred from general best practices and what we thought would make sense.

4. **Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project?**
   SFWRENG 2AA4 and SFWRENG 3RA3 are two course in particular that will help our team to be successful. These courses include requirements and software delivery which are particularly important for this course.

5. **What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project?**
   As a team, we will need to learn more about MCP and how to use it to control the user's system. We will also need to learn about LLM integration and voice to text features. We will also need to upskill in the testing domain as none of us have exhaustive knowledge on end-to-end system testing.

6. **For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Which will each team member pursue, and why did they make this choice?**
   For MCP, documentation and smaller projects are good approaches to familiarize ourselves with the technology. For LLM and Voice to speech integrations, online courses and exmaple projects on github are good real world examples we can learn from. Projects are always a good learning method as they provide hands on learning.

References

World Health Organization. Disability and health. https://www.who.int/news-room/fact-sheets/detail/disability-and-health, 2023. Accessed 2026-04-06.

World Wide Web Consortium (W3C). Web content accessibility guidelines (wcag) 2.2. https://www.w3.org/TR/WCAG22/, 2023. Accessed 2026-04-06.